# Why a Neural Pre-Decoder Fails for Surface-Code Decoding: Objective Misalignment, a Compute Roofline, and a Confidence Gate That Bounds the Damage

Nasir Ali[1*]

[1*]Embedded Systems Division, Centre for Development of Advanced Computing, Noida, 201307, India.

Corresponding author(s). E-mail(s): nasirali2607@gmail.com;

**Abstract**

Neural pre-decoders promise to cut the syndrome volume a matching decoder must resolve each surface-code round, easing a latency budget set by qubit coherence. We report a negative result with two identified causes. Training a convolutional pre-decoder on a GPU to 500,006,912 shots, over an order of magnitude beyond a prior CPU-only study, leaves it 3.79× worse than a PyMatching baseline: the loss converges, so undertraining is not the cause. The per-mechanism training objective is misaligned with logical error rate. Independently, a compute roofline on an AMD Alveo U55C bounds this architecture at 4.52 $\mu$s per shot using every DSP, so the microsecond target is unreachable by any implementation. We show a calibrated confidence gate bounds the damage, provably converging to the baseline, and quantify the headroom a perfect gate would exploit.

**Keywords:** Quantum error correction, surface code, neural decoder, FPGA, high-level synthesis, roofline analysis

## 1 Introduction

Surface-code quantum error correction requires a classical decoder to turn a stream of syndrome measurements into a correction within the time set by qubit coherence and the syndrome-extraction period [3, 4]. Minimum-weight perfect matching

(MWPM), implemented efficiently by PyMatching's sparse blossom algorithm [6, 7], is the standard choice. If decoding is slower than syndrome generation, undecoded rounds accumulate without bound, the decoder backlog problem [8, 9].

A neural *pre-decoder* is one proposed remedy: a small network resolves local detection events before an exact decoder handles the remainder [1, 2]. Recent work demonstrates real-time FPGA neural decoding on hardware [10], and GPU-resident pre-decoders reach microsecond-scale runtimes [2].

This paper reports that a faithful FPGA-oriented reimplementation of the pre-decoder concept does *not* work, and identifies why. Our contributions:

- **Scale is not the cause.** We train to 500,006,912 shots on a GPU, far beyond a prior CPU-only budget that named undertraining as its most likely failure cause. The pre-decoder remains 3.79× worse than baseline while training loss converges to 0.002197 (Section 6).
- **A compute roofline shows the latency target was never reachable.** The architecture costs 24,454,656 multiply-accumulates per shot; on the U55C's 9,024 DSPs at 300 MHz with INT8 packing, the floor is 4.52 $\mu$s per shot against a 1 $\mu$s budget (Section 4).
- **A confidence gate bounds the damage.** Escalating low-confidence shots to matching on the *raw* syndrome makes the hierarchical decoder provably no worse than the baseline, unlike an unconditional pre-decoder (Section 3).
- **An oracle bound separates gate quality from decoder value**, showing a perfect gate would reach 0.71× baseline but must find 95 useful shots among 1009 harmful ones.

## 2 Background

We simulate rotated surface-code memory circuits with Stim [5] (version 1.16.0) under circuit-level depolarising noise, and decode the resulting detector error model with PyMatching [7] (version 2.4.0). Monte Carlo collection uses `sinter` (version 1.16.0). Our target device is an AMD Alveo U55C (xcu55c-fsvh2892-2L-e), carrying 1,303,680 LUTs, 9,024 DSP slices, 2,016 block RAMs and 960 UltraRAMs across 3 super-logic regions, with 16 GB of HBM2 exposed as 32 pseudo-channels.

## 3 Design

The pre-decoder is a fully convolutional 3D network over the detector volume, with 113,358 parameters at width 64 and depth 2. It predicts which local error mechanisms fired; those corrections are applied to the syndrome, and the residual is decoded by MWPM.

### 3.1 The confidence gate

An unconditional pre-decoder has no recovery path: a wrong correction corrupts a syndrome the matcher would have decoded correctly. We therefore score each shot with seven cheap statistics (correction counts, decision margins, syndrome weight before

and after) and a calibrated logistic model, and accept the pre-decoder path only when the score reaches a threshold $\tau$. Escalated shots are decoded from the *raw* syndrome, not the corrected one. This yields two exact limits: at $\tau \to 0$ the system reduces to the unconditional pre-decoder, and at $\tau \to 1$ it reduces to the MWPM baseline. The hierarchical decoder is therefore bounded above by the baseline error rate for sufficiently large $\tau$.

# 4 The compute roofline

The architecture requires 24,454,656 multiply-accumulates per shot. Counting only these, and charging nothing for data movement, activations, control or matching, the U55C's 9,024 DSPs at 300 MHz give a floor of 4.52 $\mu$s per shot at INT8 with every DSP busy, sustaining 1.33 logical qubits. At a realistic 50 % DSP utilisation these become 9.03 $\mu$s and 0.664 qubits. The 1 $\mu$s round budget is therefore unreachable for this architecture regardless of implementation quality. Inverting the bound gives a design target: width 52 sustains one logical qubit, width 25 sustains four, and width 12 sustains sixteen (Fig. 4).

# 5 Methodology

All experiments run on an AMD EPYC 7742 64-Core Processor with 64 logical cores and an NVIDIA RTX A4000. FPGA tooling is Vitis 2023.2 with XRT 2.15.225. Results carry a provenance tier: **T1** measured on hardware, **T2** post-place-and-route, **T3** HLS estimate, **T4** simulation or analytical model. Logical error rates are Monte Carlo estimates from simulated circuits and are therefore T4; device resource counts and idle power are T1.

Baselines span 50 cells at distances 3, 5, 7, 9, totalling 164,853,021 shots and 50,576 observed logical errors, with Clopper–Pearson 95 % intervals.

# 6 Results

## 6.1 Baseline and threshold

The MWPM baseline behaves correctly: below threshold the logical error rate falls with distance and above it rises (Fig. 1). A finite-size-scaling collapse gives $p_{\mathrm{th}} = 0.695\,\%$ with bootstrap 95 % interval $[0.684, 0.706]\,\%$ and $\nu = 1.21$ over 500 converged resamples. Varying the fitting window moves the estimate over $[0.65, 0.701]\,\%$, a systematic wider than the statistical interval, so we quote both.

## 6.2 Training at scale does not help

At 500,006,912 shots (70.3 minutes at 118,524 shots/s), the training loss converges to 0.002197. Evaluated on 100,000 held-out shots at $d = 5$, $p = 0.003$, the baseline reaches 0.00328 while the unconditional pre-decoder reaches 0.01242, a factor of 3.79 worse. Because the loss has converged, this is not undertraining: the per-mechanism objective is misaligned with logical error rate, and optimising it harder makes the network apply more confident wrong corrections.

### 6.3 Capacity does not help either

Training budget is one axis; model capacity is another. We trained 3 further networks spanning widths 12 to 52, covering more than an order of magnitude in multiply-accumulates, and evaluated each identically. The logical error rate is flat: the narrowest model reaches 3.6× baseline and the widest 3.87×, a spread of only 0.27 (Fig. 3). The narrowest network is in fact marginally the best of the four.

This matters twice over. Scientifically, it is a second independent axis on which the failure does not move, which together with the training-budget result rules out both undertraining and insufficient capacity and leaves the objective as the explanation. Practically, it inverts the deployment argument: since accuracy is indifferent to width while sustainable qubits per card is not (Section 4), the cheapest network dominates. Width 12 is precisely the width the roofline identifies as sustaining sixteen logical qubits per card, against under one at the width the model was originally sized at, for no measurable accuracy cost.

### 6.4 The gate bounds the damage, and the oracle bounds the gate

Sweeping $\tau$ moves the decoder monotonically from the unconditional result to the baseline, reaching full escalation at $\tau = 0.999$. The gate is well calibrated, with expected calibration error 0.00226. An oracle that chose the better path per shot would reach 0.00233 (0.71× baseline), so the pre-decoder does carry complementary information; but it rescues only 95 shots while corrupting 1009, so exploiting that headroom demands a discrimination the present feature set cannot provide.

### 6.5 Scope and limitations of the hardware evaluation

The roofline is analytical (T4), built from T1 device counts. We report no new bitstream here; the prior implementation this work builds on measured 13–15.3 ms per shot using 331 DSPs at 100 MHz, leaving large implementation headroom that the roofline shows would still be insufficient. Logical error rates come from simulated circuits, not a quantum device.

## 7 Discussion

Two independent obstacles emerge. The accuracy failure is an objective problem: the network is trained on a proxy that does not track logical error rate. The latency failure is an architectural one: the network is too large for the device by roughly an order of magnitude. Neither is addressed by more compute, and the second is not addressed by better HLS. A loss penalising logical errors directly, and a network narrow enough to satisfy the roofline, are the natural next steps.

## 8 Conclusion

We report, and diagnose, a negative result for neural pre-decoding of the surface code on an HBM-class FPGA. Training at scale does not rescue accuracy; the architecture

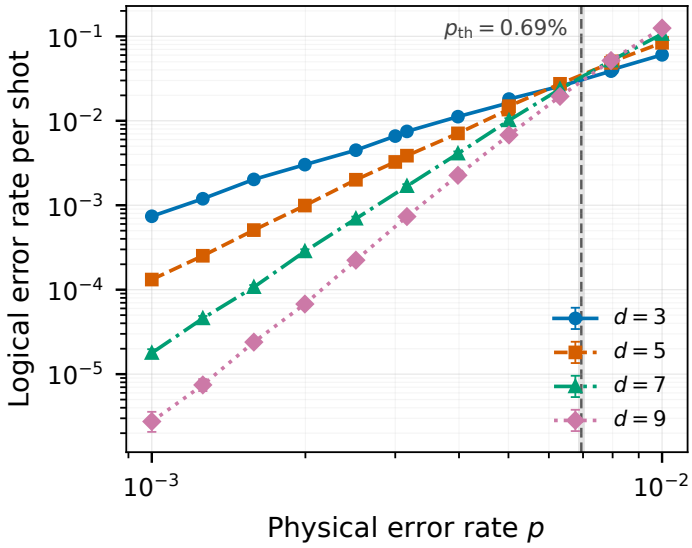


**Fig. 1** Logical error rate versus physical error rate for the MWPM baseline at distances 3, 5, 7, 9, with Clopper–Pearson 95 % intervals. The dashed line marks the fitted threshold. Provenance: T4

cannot meet the round budget on this device at any implementation quality; and a calibrated confidence gate, while unable to make the pre-decoder useful, does make it safe.

## Declarations

**Funding:** This work received no external funding.

**Competing interests:** The author declares no competing interests.

**Data availability:** All result files, figures and generated numbers are available in the public repository accompanying this paper.

**Code availability:** All code is available in the same repository.

**Author contributions:** N.A. conceived the study, implemented the software, performed the experiments and wrote the manuscript.

**Ethics approval:** Not applicable.

## References

[1] Narges Alavisamani, Suhas Vittal, Ramin Ayanzadeh, Poulami Das, and Moinuddin Qureshi. Promatch: Extending the reach of real-time quantum error correction with adaptive predecoding, 2024.

[2] Christopher Chamberland, Jan Olle, Muyuan Li, Scott Thornton, and Igor Baratta. Fast and accurate ai-based pre-decoders for surface codes, 2026.

[3] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. Topological quantum memory. *Journal of Mathematical Physics*, 43(9):4452–4505, 2002.

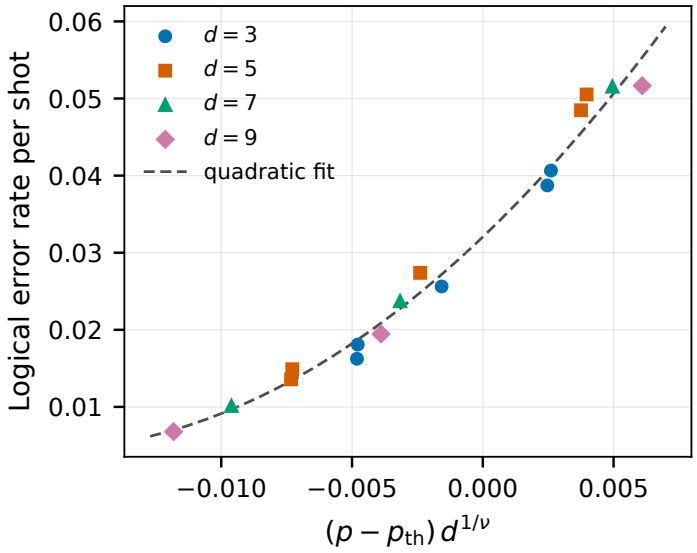


**Fig. 2** Finite-size-scaling collapse of the baseline data onto a single quadratic. Provenance: T4

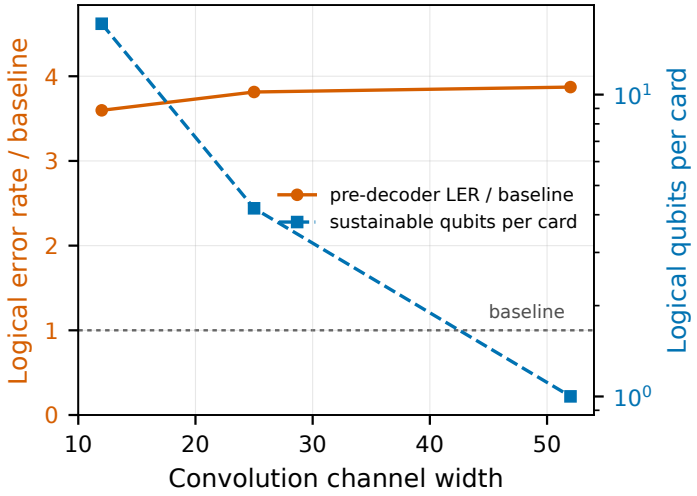


**Fig. 3** Logical error rate relative to baseline (left axis) and sustainable logical qubits per card (right axis) against convolution width. Accuracy is flat while deployability varies by more than an order of magnitude. Provenance: T4

[4] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3), 2012.

[5] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021.

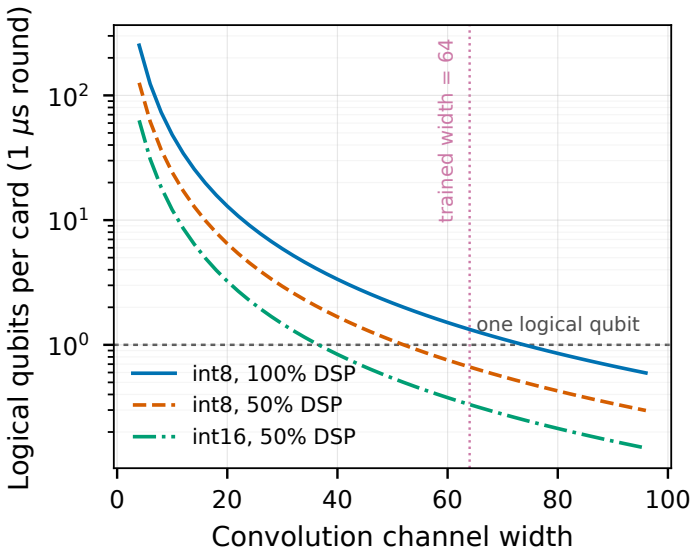


**Fig. 4** Sustainable logical qubits per card versus convolution width, from the compute roofline. Provenance: T4 bound from T1 device counts


[6] Oscar Higgott. Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching, 2021.

[7] Oscar Higgott and Craig Gidney. Sparse blossom: correcting a million errors per core second with minimum-weight matching. *Quantum*, 9:1600, 2025.

[8] Luka Skoric, Dan E. Browne, Kenton M. Barnes, Neil I. Gillespie, and Earl T. Campbell. Parallel window decoding enables scalable fault tolerant quantum computation. *Nature Communications*, 14(1), 2023.

[9] Barbara M. Terhal. Quantum error correction for quantum memories. *Reviews of Modern Physics*, 87(2):307–346, 2015.

[10] Xiaohan Yang, Xuandong Sun, Zhiyi Wu, Jiawei Zhang, Ji Jiang, Xiayu Linpeng, Yuxuan Zhou, Ji Chu, Jingjing Niu, Youpeng Zhong, Song Liu, and Dapeng Yu. Real-time surface-code error correction using an fpga-based neural-network decoder, 2026.